

1. Several Variations on Gradient Descent Method to Improve the Learning Processes:

A. SGD with learning rate, momentum, nesterov, decay:

⇒ this combination enhances pure SGD by using momentum to accelerate the learning process and using decay to stabilize late-stage convergence.

⇒ momentum: combine the gradient of current position with the gradients for several previous weights adjustment to get the gradient for current weights adjustment.

for example, if the gradient of current position is 2, accumulated past momentum ($\equiv$ the gradients for several previous weights adjustment) is 10, and $\text{momentum} = 0.6$, and then we can receive the gradient descent for current weights adjustment $= -[2 + (0.6 \cdot 10)] = -8$.

⇒ nesterov: another type of smarter momentum that use the current position, momentum, and accumulated past momentum ($\equiv$ the gradients for several previous weights adjustment) to project ($\equiv$ predict) the future position first, and then seeing the gradient of this future position as the gradient of current position to calculate the gradient for current weights adjustment by the momentum mentioned above.

for example, if $\text{nesterov} = \text{true}$, current position = 50, accumulated past momentum ($\equiv$ the gradient for several previous weights adjustment) is 10, and $\text{momentum} = 0.6$,

and then we can project future position $= 50 - (0.6 \cdot 10) = 44$ first, the gradient descent of future position (44) is 0.5, and we can get the gradient descent for current weights adjustment $= -[0.5 + (0.6 \cdot 10)] = -6.5$.

⇒ decay: it can gradually shrink the value of learning rate as training iterations increase during learning processes to ensure stable convergence.

for example, if $\text{decay} = 0.01$, initial learning rate = 0.1, and then we can receive the learning rate in k -th iteration $= (0.1) / (1 + 0.01 \cdot k)$.

B. Adaptive Gradient (AdaGrad) with learning rate, epsilon:

⇒ this function internally executes the standard gradient descent method, but the learning rate is dynamically adaptive by automatically shrinking the learning rate.

⇒ epsilon (ϵ): a tiny constant added to the denominator to prevent a mathematical crash cause by dividing by zero when dynamically adjusting the learning rate

$$\text{new learning rate} = \frac{\text{old learning rate}}{\sum (\text{the gradient for several previous weights adjustment})^2 + \epsilon}$$

G_t

if old learning rate is 0.1, and $G_t = 0$. With epsilon (ϵ) = 10^{-7} , new learning rate can be calculate $\frac{0.1}{\sqrt{0 + 10^{-7}}}$ safely.

C. RMSProp with learning rate, epsilon, rho:

⇒ this function internally executes the standard gradient descent method, but the learning rate is dynamically adaptive by automatically shrinking the learning rate.

furthermore, it solves AdaGrad flow of shrinking the learning rate too fast by using a more efficient adaptive-learning rate method.

⇒ rho (ρ): the rate that determines how much $\sum (\text{the gradients for several previous weights adjustment})^2$ versus how much $(\text{the gradient of current position})^2$ influence the gradient for current weights adjustment.

for example, if $\rho = 0.9$, and then we can get the gradient for current weights adjustment which influenced by $\sum (\text{the gradient for several previous weights adjustment})^2$ for 90% and influenced by $(\text{the gradient of current position})^2$ for 10%.

D. Adaptive Moment Estimation (Adam) with learning rate, epsilon, decay:

⇒ this function internally executes the standard gradient descent method, but the learning rate is dynamically adaptive by automatically shrinking the learning rate.

furthermore, it act as a hybrid by combining the benefits of both the momentum and RMSProp.

2. Exploding Gradients and Gradients Clipping:

A. exploding gradients: a problem that the gradients become excessively large at some points during the backward pass

this causes massive and unstable step size during weights adjustment, which can completely throw off the neural network and ruin the learning process.

B. gradients clipping: a technique utilized to avoid the issue of exploding gradients by simply clipping the gradients with overly large values so that those values aren't allowed during weigh adjustment.

for example, assume the maximum gradient limit is set to 5,

⇒ normal scenario: the calculated gradient is 3. since $3 < 5$, the optimizer uses 3 to tune the weights.

⇒ exploding scenario: a sudden spike causes the calculated gradient to reach 500. since $500 > 5$, the clipping mechanism triggers and forces the value 500 $\rightarrow$ the maximum threshold of 5, protecting the neural network from destruction.